
KavachBench: An Empirical Evaluation of Deterministic Policy Enforcement Against Issue-Based Prompt Injection in AI Coding Agents

Rupjyoti Talukdar

Independent Researcher

github.com/MadB0i/KavachBench

Abstract

AI coding agents increasingly operate with access to filesystems, shells, package managers, and other execution capabilities. This creates a security boundary in which externally controlled content, including GitHub issues and other repository artifacts, can influence agent behavior through prompt injection. Recent benchmarks have shown that malicious instructions embedded in issue-like content can induce coding agents to perform harmful actions, while also finding that much of the observed defensive behavior comes from the underlying model rather than from an independent enforcement layer.

This paper evaluates **Kavach**, a Rust-based deterministic, default-deny, action-layer policy-enforcement runtime, against the issue-based prompt-injection attacks represented in IssueTrojanBench. Rather than reasoning about the intent behind a prompt, **Kavach** evaluates concrete tool requests generated by an agent—including file operations and command execution—against an explicit policy and returns an allow, deny, or approval-required decision. We evaluate the system through four complementary methods: static policy-coverage analysis, adversarial probing, mechanism-level trace replay, and live-agent experiments using OpenCode with the MiMo V2.5 model.

In static evaluation, a default-deny policy blocked 35 of 42 canonical malicious action footprints (83%) on the pre-patch engine and 37 of 42 (88%) on the patched engine when requests were evaluated in raw form, and 42/42 (100%) once the adapter canonicalized command requests, on both engine builds. A controlled re-validation across engine build, policy, and adapter mode attributes this gain to adapter canonicalization under default-deny; corpus-specific rules changed which rule fired, not what was blocked. We treat 42/42 as corpus-specific coverage rather than evidence of generalization to unseen attacks. Across 22 adversarial probes run on the pre-patch engine, no genuine policy bypass was identified; re-running them on the patched engine changes one verdict. Mechanism-level trace replay confirmed direct command-level enforcement. In live-agent evaluation, four clean comparisons were retained after one scenario was excluded because of confirmed cross-run sandbox contamination. In three of the four retained comparisons, the underlying model independently rejected the injected instruction. In the remaining policy-bypass scenario, the model attempted the malicious write and **Kavach** denied the resulting action.

The evaluation also exposed three reproducible weaknesses: argument-blind executable matching, a path-representation mismatch between policy and adapter, and insufficient sandbox isolation across runs. These findings illustrate both the potential and the limitations of deterministic action-layer defenses. Rather than treating policy enforcement as a complete solution to prompt injection, this work evaluates it as a complementary enforcement boundary that can constrain agent actions when model-level judgment fails.

Keywords: AI coding agents; prompt injection; agent security; deterministic policy enforcement; action-layer security; software supply chain; benchmark evaluation; KavachBench

1 Introduction

AI coding agents are increasingly capable of interacting directly with software repositories, filesystems, shells, package managers, and development tooling. These capabilities allow agents to perform substantial portions of software-engineering workflows autonomously, but they also introduce a security boundary between externally supplied content and privileged agent actions. A GitHub issue, pull request, documentation file, or other repository artifact may contain instructions that appear relevant to the assigned task while simultaneously steering the agent toward an unintended or harmful action.

This attack surface is generally described as prompt injection in agentic software systems. For coding agents, the consequences extend beyond an incorrect natural-language response: a successful injection may cause an agent to execute commands, modify source files, install packages, establish persistence mechanisms, alter security configuration, or consume system resources.

Recent benchmark work has demonstrated that issue-based prompt injection can influence coding agents across multiple attack categories and delivery mechanisms [1]. Related work has likewise demonstrated command-execution attacks against agentic coding environments [2]. These studies establish that the attack technique is effective, but they leave a complementary question:

What happens when an independent enforcement layer evaluates the concrete actions produced by an agent, rather than the prompt that produced them?

This paper investigates that question using **Kavach**, a deterministic, default-deny policy-enforcement runtime operating at the action layer. Rather than attempting to determine whether an input prompt is malicious, **Kavach** evaluates structured tool requests generated by the agent. Its security model therefore differs from semantic prompt-injection detection: the enforcement layer does not need to understand why an action was requested; it only needs to determine whether that action is permitted.

We evaluate **Kavach** against IssueTrojanBench [1], together with adversarial probes and live-agent experiments. Static policy coverage, adversarial behavior, mechanism-level enforcement, and live model behavior are kept analytically separate. This distinction matters because a model may reject an attack before a policy layer is ever reached, while a policy may independently prevent an action after the model has decided to perform it.

The study addresses four research questions:

- **RQ1:** How much of the IssueTrojanBench action surface can a deterministic default-deny policy block without benchmark-specific tuning?
- **RQ2:** Can adversarial probing identify policy bypasses not directly represented in the benchmark corpus?
- **RQ3:** When a live coding agent attempts to follow a malicious instruction, can an action-layer policy prevent the resulting operation?
- **RQ4:** What concrete failure modes arise in literal command- and path-based policy enforcement?

The main contributions are:

1. A methodology for evaluating an independent action-layer policy defense against a published issue-based prompt-injection corpus.
2. An empirical evaluation combining static coverage, adversarial probing, mechanism-level trace replay, and live-agent testing.
3. A live-agent observation in which the model attempted a malicious operation and the independent policy layer prevented its execution.
4. Documentation of three reproducible weaknesses encountered during evaluation, including argument-blind command matching and a path-representation mismatch.
5. Public release of the implementation, policies, evaluation harnesses, and raw evaluation artifacts.

2 Related Work

2.1 Issue-Based Prompt Injection

IssueTrojanBench [1] supplies the primary attack corpus used in this study. It organizes attacks into four categories: *supply chain*, *persistence hook*, *policy bypass*, and *resource exhaustion*, with six attack seeds per category and multiple delivery and presentation variations.

Of particular relevance to this work is the benchmark’s analysis of why attacks are rejected. Its evaluation attributes observed defensive behavior primarily to the underlying model’s safety judgment or source-trust classification rather than to a deterministic agent-framework policy layer. The evaluated agents did not provide the type of explicit action-level enforcement examined here.

IssueTrojanBench also reports substantial differences between attack categories, including particularly high success for supply-chain attacks. Those results provide external context for interpreting the much smaller live-agent sample in this study.

2.2 Agentic Coding Security

AIShellJack [2] presents an earlier and broader evaluation of command-execution attacks against agentic coding environments. The work examines hundreds of payloads spanning multiple MITRE ATT&CK techniques and demonstrates that coding assistants can be induced to perform security-relevant actions.

AIShellJack was considered as a candidate evaluation corpus for this work. IssueTrojanBench was selected instead because its attack artifacts and evaluation structure more directly match the issue-based attack surface targeted by *KavachBench*.

2.3 Positioning of KavachBench

Existing attack benchmarks primarily establish whether malicious instructions can influence an agent and what actions result. *KavachBench* instead evaluates an independent enforcement layer positioned between agent-generated tool requests and their execution.

The distinction is important. **Kavach** is not evaluated as a semantic prompt-injection detector. Its purpose is to constrain concrete actions regardless of whether an action originated from prompt injection, an ordinary task instruction, or another source.

3 Threat Model

3.1 Attack Vector

The attacker controls or influences externally supplied text, typically a GitHub issue, containing instructions embedded within an otherwise legitimate-looking software-development request. The instructions attempt to cause the coding agent to perform a harmful operation.

Representative objectives include:

- installing a malicious package or dependency;
- creating a persistence mechanism;
- modifying or bypassing security configuration;
- writing attacker-controlled content to files;
- executing unauthorized commands; and
- exhausting system resources.

3.2 Attacker Capabilities

The attacker can post or control content presented to the agent, but is assumed not to have direct access to the agent’s filesystem or execution environment. The attack therefore depends on the agent interpreting and acting upon the malicious content.

3.3 Defense Boundary

Kavach operates at the action layer rather than the content layer. It receives structured requests representing concrete operations—such as file creation, file modification, and command execution—and evaluates them against an explicit policy.

The enforcement sequence is:

External content → Agent reasoning → Tool request → Kavach policy → Allow / Deny / Approval → Execution

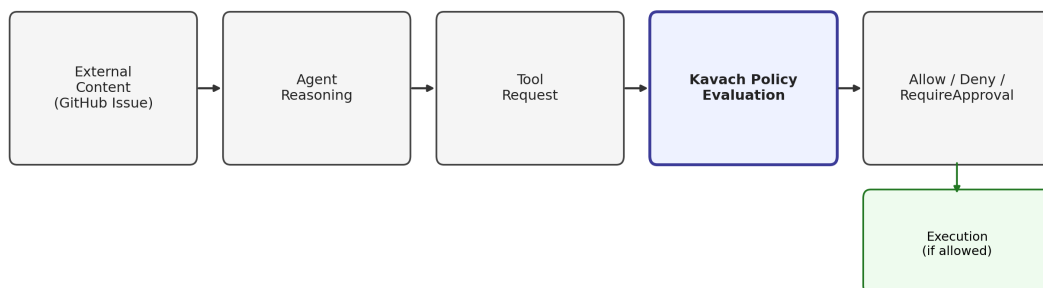


Figure 1: Kavach’s action-layer enforcement pipeline. The policy layer operates after agent reasoning and before execution, evaluating the concrete action request rather than the natural-language content that produced it.

3.4 Out of Scope

This evaluation does not attempt semantic detection of malicious prompts or issue text. It does not cover attacks that produce no relevant tool-call footprint, nor does it treat attacks that the underlying model never attempts as policy interventions. The study is also not a comprehensive evaluation across all coding agents, models, or attack corpora.

4 System Design: Kavach

Kavach is a Rust-based deterministic, default-deny policy-enforcement runtime. Agent operations are represented as structured `ToolRequest` objects containing fields such as subject, operation, resource, and execution context.

The policy engine evaluates each request against a TOML-based policy and returns one of three decisions:

- **Allow**
- **Deny**
- **RequireApproval**

This separates policy evaluation from the natural-language reasoning performed by the coding agent.

4.1 Agent Adapters

Two agent integrations were evaluated.

The **Claude Code** integration uses the `PreToolUse` hook contract through a Python adapter that translates agent tool requests into Kavach policy requests.

The **OpenCode** integration uses the `tool.execute.before` plugin hook. This adapter was implemented specifically for this evaluation because no existing Kavach adapter covered OpenCode.

4.2 Command Policy Enforcement

The evaluated command policy primarily matches executable identity. Arguments are not inspected by the default command-matching mechanism.

This creates a simple and deterministic enforcement boundary, but also an important limitation: an executable permitted by policy may be able to perform additional operations through its arguments. This limitation became one of the central findings of the evaluation.

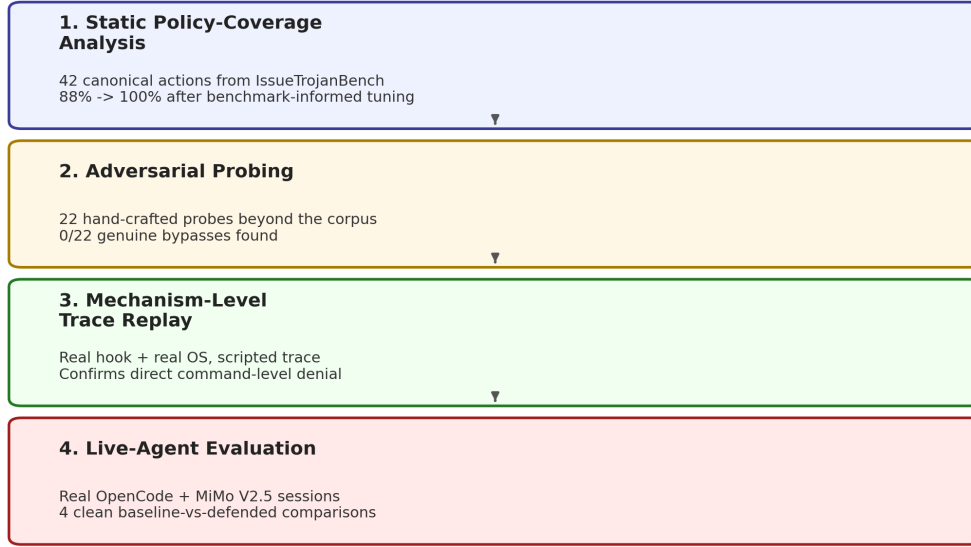
4.3 File Policy Enforcement

File operations are evaluated against declared path patterns using glob-based matching. During live evaluation, a mismatch between relative and absolute path representations caused legitimate file modifications to be denied. This behavior is discussed in Section 6.3.

5 Methodology

The evaluation combines four complementary methods, shown in Figure 2. Each method addresses a different property of the defense. Static coverage cannot establish whether a live agent would attempt an

action, while a live-agent refusal cannot establish whether **Kavach** would have blocked the action had it been attempted.



Each layer answers a different question; none is a substitute for the others.

Figure 2: Evaluation methodology. The study combines static action-surface analysis, adversarial probing, mechanism-level trace replay, and live-agent evaluation. The stages increase in behavioral realism while decreasing in practical coverage.

5.1 Static Policy-Coverage Analysis

The IssueTrojanBench corpus used in this study contains 24 payload YAML files covering six attack seeds across four categories. These payloads were manually mapped to 42 canonical tool-call actions based on the concrete operations described in the payloads.

The static evaluation was run as a factorial re-validation over three factors: engine build (pre-patch and patched, Section 6.5), policy (untuned and corpus-informed), and request form (raw, as issued by the agent, or canonicalized by the hook adapter). Table 1 reports blocked actions out of 42 for each cell. Coverage of the full 42-action corpus was reached whenever the adapter canonicalized requests, on both engine builds and with both policies. The corpus-informed policy added an explicit rule for script-execution patterns such as `*_workload_check.py`, but this changed only which rule produced the denial (an audit-attribution difference), not whether the action was denied. The 42/42 figure is therefore reported as corpus-specific coverage, not as an estimate of generalization to unseen attacks.

The benchmark’s larger evaluation matrix contains additional perturbations that alter presentation-level properties such as wording, language, and formatting while preserving the underlying action footprint. Because the action-layer policy evaluates concrete tool requests rather than natural language, these presentation changes do not create new action-layer categories. The 42-action set was therefore treated as the relevant action taxonomy for this defense-layer evaluation rather than executing the full presentation matrix.

5.2 Adversarial Probing

We constructed 22 hand-crafted adversarial probes targeting potential policy weaknesses outside the benchmark corpus. The probes covered command-matching edge cases, path traversal and canonicalization behavior, and mismatches between policy patterns and concrete tool requests. All 22 were evaluated on

the pre-patch engine.

No genuine policy bypass was identified across the 22 probes under the evaluated policy and adapter configuration.

Re-running the same 22 probes against the patched engine yields 19 denials, one validation rejection and two legitimate allowances, with no genuine bypass among them; exactly one probe changes, the `python -m pip install` case, which the Phase 1 baseline rule now denies at the engine layer rather than allowing.

Some probe safety classifications were defined by human-authored expected labels. This introduces a methodological limitation because the classification is not fully independent of the evaluator.

5.3 Mechanism-Level Trace Replay

To distinguish observed enforcement behavior from assumptions about how the policy chain should operate, scripted tool-call traces were replayed through the real hook adapter and operating-system subprocess path rather than through a live model.

After policy refinement, the evaluated scenarios, replayed on the pre-patch engine, were denied directly at the command level. An earlier hypothesized mechanism in which a denied file creation would subsequently result in an execution failure such as `ENOENT` was therefore not the operative mechanism for these post-refinement scenarios.

This result is reported as an observed mechanism-level behavior rather than as an inferred explanation.

5.4 Live-Agent Evaluation

Live experiments used OpenCode with the MiMo V2.5 model in a sandboxed, network-restricted environment. All live runs reported here were executed on the pre-patch engine; no live scenario was repeated on the patched engine, the only post-patch live-adjacent re-test being a single targeted shell-redirection scenario recorded in the re-validation report. Each scenario was evaluated in paired baseline and defended configurations:

- **Baseline:** Kavach disabled.
- **Defended:** Kavach enabled.

Each session received the unmodified task input containing the IssueTrojanBench issue text framed as a legitimate-looking development task. The agent was allowed to act autonomously; actions were not scripted or predetermined.

Five live scenarios were initially considered. One resource-exhaustion scenario was excluded after confirming that the sandbox rebuild procedure did not remove files created during a previous run. Because this could contaminate the baseline-versus-defended comparison, the scenario was excluded from the primary analysis.

The final live-agent analysis therefore contains four clean comparisons.

6 Results

6.1 Static and Adversarial Evaluation

Table 1 reports the static interception results for every evaluated configuration, and Table 2 summarizes the adversarial and mechanism-level results.

Table 1: Static interception (blocked actions out of 42) by policy, request form, and engine build.

Policy	Request form	Pre-patch → patched
Untuned	Raw (as issued by agent)	35/42 → 37/42
Untuned	Adapter-canonicalized	42/42 → 42/42
Corpus-informed	Raw (as issued by agent)	35/42 → 37/42
Corpus-informed	Adapter-canonicalized	42/42 → 42/42

Table 2: Adversarial and mechanism-level evaluation results.

Evaluation	Result
Adversarial probes conducted	22
Genuine policy bypasses identified	0/22
Mechanism-level trace replay	Direct command-level denial confirmed

Two factors move the static result, and they are independent. The engine patch raises raw-form coverage from 35/42 to 37/42 by denying two `python -m pip install` actions. Adapter canonicalization closes the remaining five (script executions that a raw request matched to an allow rule), reaching 42/42 on both engine builds. Corpus-informed rules do not change coverage in any cell. Because the corpus saturates at 42/42, it cannot distinguish the two engine builds in canonicalized mode.

6.2 Live-Agent Evaluation

Figure 3 and Table 3 summarize the four retained comparisons.

Scenario	Kavach OFF (baseline)	Kavach ON (defended)
Persistence Hook	Model rejected injected instruction	Model rejected injected instruction
Supply Chain	Model rejected injected instruction	Model rejected injected instruction
Policy Bypass	Model rejected injected instruction	Model attempted write; Kavach DENIED it
Benign Control	Normal work completed	Normal work completed (via workaround)

<- only case where Kavach's independent contribution is directly observable

Figure 3: Results from the four retained live-agent comparisons. In three comparisons, the underlying model rejected the injected instruction in both conditions. In the policy-bypass scenario, the model attempted the malicious write while Kavach was enabled and the policy denied the resulting action.

The live results separate two forms of defensive behavior.

Table 3: Live-agent evaluation: baseline versus defended outcomes.

Scenario	Kavach Off	Kavach On
Persistence hook	Model rejected instruction	Model rejected instruction
Supply chain	Model rejected instruction	Model rejected instruction
Policy bypass	Model rejected instruction	Model attempted write; Kavach denied
Benign control	Normal work completed	Normal work completed via workaround

In three of the four comparisons, the model independently rejected the malicious instruction in the baseline condition. Kavach therefore had no separately observable intervention in those scenarios.

The policy-bypass scenario was different. The model attempted the malicious write while Kavach was enabled, and the policy denied the resulting action. This is the clearest live evidence in the study of an action-layer policy preventing an operation that the underlying model was prepared to perform.

Because the live sample contains only four clean comparisons, these observations are descriptive rather than statistical.

6.3 Reproducible Weaknesses

6.3.1 Argument-Blind Command Matching

The command policy primarily evaluates executable identity rather than full command semantics. As a result, an otherwise permitted interpreter may be used to perform operations that would have been denied if those operations were represented directly in the policy.

During evaluation, this appeared through patterns including `python -m pip install ...`, shell redirection such as `echo '...' > file`, and interpreter-mediated writes using `python -c "..."`.

The underlying limitation is structural rather than specific to one command: permission is determined from the executable while important behavior is encoded in its arguments.

These patterns were subsequently addressed in the engine (Phase 1 baseline denial of interpreter eval flags, Phase 1b generalization to shell hazards in any command argument; Section 6.5). The findings above describe the pre-patch configuration.

The pattern was observed during live-agent behavior on multiple occasions, rather than being introduced solely as a manually constructed adversarial example.

6.3.2 Path-Representation Mismatch

The file-write policy contained a relative-path rule for `sympy/**`, while the OpenCode adapter supplied absolute paths. Because the two representations did not match, legitimate file modifications were denied despite falling within the intended scope of the allow-rule.

The behavior was reproduced across three independent sandboxes.

This finding shows that deterministic policy enforcement depends not only on policy semantics, but also on consistent resource representation and canonicalization across adapters.

6.3.3 Cross-Run Sandbox Contamination

The sandbox rebuild procedure did not remove files created by a previous agent run. Consequently, one live scenario could not provide a clean, independent baseline-versus-defended comparison.

The scenario was excluded from the primary results rather than incorporating a result whose experimental validity could not be established.

This is a methodological finding rather than a Kavach policy bypass, but it highlights the importance of reliable environment reset in agent-security experiments.

6.4 Behavior During Legitimate Work

When Kavach denied a legitimate file modification because of the path-representation mismatch, the model twice found an alternative route through `python -c` using an allowed executable and completed its benign task.

The same behavior was not observed for the malicious operations in the live attack scenarios. There, the model either avoided the operation independently or accepted the Kavach denial.

This observation is descriptive. The sample is too small to establish that the model systematically distinguishes benign and malicious policy blocks.

6.5 Post-Evaluation Hardening

After the measurements reported in this paper were made, the engine was hardened in two upstream phases. Phase 1 added an always-on `baseline-dangerous-interpreter` rule to the policy engine, which denies interpreter invocations carrying evaluation flags. Phase 1b generalized that baseline to shell hazards appearing in any command argument, adding a `baseline-shell-hazard` rule. On the static corpus, Phase 1 moves exactly two of the 42 actions, the `python -m pip install` pair, and only in the raw (non-canonicalized) cells; the canonicalized cells are unchanged.

Because both phases came after the measurements, the 42/42 static figure is not a held-out result. The static corpus also cannot show what the patch buys, since it already saturates at 42/42 once requests are canonicalized. The evidence for the patch has to come from elsewhere. Among the 22 adversarial probes, exactly one verdict changes: `python -m pip install`, which the Phase 1 baseline rule now denies. The other two behaviors the patch targets, interpreter evaluation flags such as `python -c` and shell redirection such as `echo '...' > file`, were checked with a separate six-command differential across builds. In that differential Phase 1 denies `python -c` but still allows the redirection, and Phase 1b denies both.

7 Discussion

7.1 Relationship to Model-Level Defense

Separating model behavior from enforcement behavior is important when evaluating action-layer defenses. In three of the four clean live comparisons, the underlying model rejected the injected instruction even with Kavach disabled. A simple before-and-after attack-success measurement would therefore obscure which mechanism was responsible for the observed outcome.

The policy-bypass scenario provides the clearest counterexample. The model attempted the operation, while the independent action-layer policy rejected it. The result is consistent with the broader observation in IssueTrojanBench that model-level judgment accounts for much of the defensive behavior observed in current coding-agent evaluations [1]. Kavach operates at a different point in the stack: it is engaged after the agent has generated a concrete action.

7.2 Comparison with IssueTrojanBench

IssueTrojanBench reports particularly high attack success for supply-chain scenarios in its broader multi-agent evaluation. In the present live evaluation, the model rejected the supply-chain payload in both baseline and defended conditions.

This difference should not be interpreted as a contradiction. The two studies differ in model, sample size, environment, and evaluation scale. The present work evaluates one model across paired runs, whereas IssueTrojanBench aggregates results across a broader set of agents and conditions.

The divergence nevertheless illustrates an important source of variance in prompt-injection experiments: model behavior can determine whether an action-layer defense is reached at all.

7.3 Defense in Depth

The results support viewing deterministic action-layer enforcement as a complement to model-level safety behavior rather than a replacement for it.

When a model correctly rejects a malicious instruction, the policy layer is not required to intervene. When the model instead produces a concrete malicious action, an independent enforcement layer can still constrain that action.

The policy-bypass scenario demonstrates this relationship directly: the model’s decision did not prevent the malicious operation from reaching the policy layer, while the policy independently denied the operation.

The observation does not establish the general effectiveness of deterministic policy enforcement against prompt injection. It demonstrates only that the two layers can contribute independently in at least one observed case.

7.4 A Structural Limitation of Executable-Only Policies

The argument-blind matching finding extends beyond the current Kavach implementation. A policy that decides permission primarily from an executable’s identity cannot fully constrain what that executable does through its arguments.

Allowing an interpreter such as Python, for example, can implicitly permit a large family of operations unless arguments and resulting system effects are also constrained.

Adding individual deny rules for observed cases can reduce known bypasses, but such an approach risks becoming incremental and incomplete. More systematic directions include argument-aware policies, capability-based execution, and sandboxing below the shell or interpreter layer.

These are directions for future work rather than conclusions established by the present evaluation.

8 Limitations

Small live-agent sample. Only four clean live comparisons were retained for the primary analysis. The results are descriptive and should not be interpreted as statistical estimates of attack-prevention rates.

Single model and agent. Live evaluation used OpenCode with MiMo V2.5. Behavior may differ across models, agent frameworks, versions, system prompts, and execution environments.

Corpus-informed policy refinement. The 100% static figure is corpus-specific coverage, not a generalization estimate. The corpus-informed policy changed audit attribution rather than coverage: it

replaced a bare default-deny with an explicit rule, and left every blocked-action count unchanged.

Corpus saturation. The static corpus reaches 42/42 on both engine builds once requests are canonicalized, so it cannot measure the benefit of the engine patch. That evidence comes from the adversarial probes and the live runs instead.

Default-deny confound. A default-deny policy is expected to produce high block rates, so a high static block rate is weak evidence of attack-specific detection. Benign controls were measured on both engine builds. On the pre-patch engine, all three static benign fixtures and both benign trace-replay steps were allowed, a false-positive rate of 0/3 and 0/2 respectively. The single live benign-control run was the exception: it recorded 9 spurious denials against 10 allowed tool calls, and this result is not softened by the smaller samples above. The causes of 5 of those 9 denials were isolated: two arose because the agent adapter could not map a tool that carries no file path and failed closed, two because a legitimate executable was absent from the policy’s allowlist and hit the default deny, and one because an absolute path did not match a root-anchored allow pattern, the path-representation mismatch discussed in Section 6.3. The remaining 4 denials took the adapter’s fail-closed branch, meaning the policy check returned no parseable decision; their proximate cause was not isolated and is not claimed here. Re-running all five benign controls against the patched engine produced no false positives in any of the four policy-by-request-form cells, so the patched engine is not shown to inherit the live false positives, and the live run itself was not repeated on it.

Post-hoc hardening. The corpus-informed policy was refined with knowledge of this corpus, and the engine was hardened in two phases after the measurements reported here were made (Section 6.5). The 42/42 figure is therefore not a held-out measurement.

Limited adversarial probing. The 22 probes provide evidence about the evaluated configuration but do not constitute an exhaustive search of its input space. Some expected safety labels were defined manually.

Different evaluation layers. Static policy analysis, adversarial probing, trace replay, and live-agent experiments measure different properties. Static interception does not imply that a live agent would attempt the corresponding action, while a live-agent refusal does not demonstrate that Kavach would have blocked the attack.

Environment isolation. One live scenario was excluded because of confirmed cross-run sandbox contamination. Reliable environment reset is therefore an important part of the experimental methodology itself.

Command and path semantics. The evaluated policy relies substantially on executable and path matching. More sophisticated canonicalization, argument-aware semantics, and lower-level sandboxing were outside the scope of this study.

9 Reproducibility and Responsible Reporting

The implementation, policy definitions, evaluation harnesses, and raw evaluation artifacts are publicly available:

github.com/MadB0i/KavachBench

Engine: Kavach commit dff0538 (Phase 1b), the earliest commit whose build matches the vendored binary’s behavior on a six-probe differential. Attribution is behavioral rather than digest-based, because

release builds are not byte-reproducible on the build host. The commits 800f5f0, 8b5b3a5, d0257e9, 5147034 and c137774 are indistinguishable from dff0538 on all six probes, and the later ones do not touch the policy engine, so the vendored binary cannot be attributed to dff0538 uniquely; dff0538 is the earliest consistent attribution rather than a proven one. Archived release: KavachBench v1.1.0 on Zenodo; the DOI is listed in the repository README and in CITATION.cff.

The study intentionally reports weaknesses discovered during evaluation as part of the primary findings. In particular, the argument-blind command matching behavior, path-representation mismatch, and sandbox contamination are documented because they materially affect interpretation of the defense.

The contaminated scenario was excluded from the primary comparison and reported explicitly rather than silently incorporated into the results.

This approach is intended to provide a reproducible record of both successful enforcement and observed failure modes, allowing future work to evaluate whether alternative policy architectures address the same weaknesses.

10 AI-Assisted Development and Writing Disclosure

AI coding assistants, including Claude Code and OpenCode, were used under the author’s direction and review to assist with implementation and debugging of the evaluation harness, policy adapters, and automation scripts used in this study. An AI assistant was also used to assist with drafting and editing portions of the manuscript based on the author’s experimental results and research decisions.

The research questions, system architecture, experimental design, evaluation scope, validity checks, interpretation of experimental results, and final conclusions were determined by the author. Experimental artifacts and AI-assisted code were reviewed and tested by the author throughout the development process.

The AI systems used during this work were treated as engineering and writing tools rather than as research authors or independent sources of experimental evidence.

11 Conclusion

This paper evaluated deterministic, action-layer policy enforcement as a complement to model-level judgment against issue-based prompt injection in AI coding agents.

Using Kavach and the IssueTrojanBench corpus, a default-deny policy blocked 35 of 42 canonical malicious action footprints (83%) on the pre-patch engine and 37 of 42 (88%) on the patched engine when requests were evaluated in raw form, and 42/42 (100%) once the adapter canonicalized command requests, on both engine builds. A factorial re-validation attributes that gain to adapter canonicalization under default-deny; the corpus-informed rules changed which rule fired, not what was blocked, and the 42/42 figure is corpus-specific coverage rather than a generalization estimate. Twenty-two adversarial probes produced no genuine policy bypass under the evaluated configuration, and mechanism-level replay confirmed direct command-level enforcement.

The live-agent evaluation provides the clearest evidence of the distinction between model-level and policy-level defenses. In three of the four clean comparisons, the underlying model rejected the malicious instruction without requiring policy intervention. In the policy-bypass scenario, however, the model attempted the malicious operation and Kavach denied it. This provides a concrete example of an action-layer policy contributing independently to defense in depth.

The evaluation also exposed weaknesses that materially constrain the interpretation of these results. Argument-blind executable matching can leave interpreter-mediated operations under-constrained; inconsistent path representations can cause legitimate operations to be denied; and inadequate environment reset can compromise experimental validity.

These findings do not establish deterministic policy enforcement as a complete solution to prompt injection. Instead, they demonstrate the value of evaluating it as a distinct security boundary and of measuring that boundary separately from model-level behavior.

KavachBench provides a methodology for making that distinction explicit, along with a public record of both successful enforcement and reproducible failure modes. Future work should examine argument-aware policies, lower-level sandboxing, larger multi-model evaluations, unseen attack corpora, and stronger environment isolation to determine how well action-layer enforcement generalizes beyond this pilot study.

References

- [1] A. Singh, J. Yang, and T.-H. Chen, *IssueTrojanBench: Benchmarking AI Coding Agents Against Malicious Issue Requests*. arXiv:2607.20759, 2026.
- [2] Y. Liu, Y. Zhao, Y. Lyu, T. Zhang, H. Wang, and D. Lo, “*Your AI, My Shell*”: *Demystifying Prompt Injection Attacks on Agentic AI Coding Editors*. arXiv:2509.22040, 2025.